```sql
-- ########################################################################
-- Red social para la comunidad estudiantil pascualina
-- ########################################################################

DROP SCHEMA IF EXISTS red_social CASCADE;
CREATE SCHEMA red_social;
SET search_path TO red_social;

-- ########################################################################
-- 1. ENTIDADES FUERTES
-- ########################################################################

CREATE TABLE estudiante (
    id_estudiante        INTEGER GENERATED ALWAYS AS IDENTITY,
    nombre               VARCHAR(60)  NOT NULL,
    apellido             VARCHAR(60)  NOT NULL,
    correo_institucional VARCHAR(120) NOT NULL,
    carrera              VARCHAR(100) NOT NULL,
    semestre             SMALLINT     NOT NULL,
    fecha_nacimiento     DATE         NOT NULL,
    biografia            VARCHAR(500),
    CONSTRAINT pk_estudiante PRIMARY KEY (id_estudiante),
    CONSTRAINT uq_estudiante_correo UNIQUE (correo_institucional),
    CONSTRAINT ck_estudiante_semestre CHECK (semestre BETWEEN 1 AND 12),
    CONSTRAINT ck_estudiante_nacimiento CHECK (fecha_nacimiento <
CURRENT_DATE),
    CONSTRAINT ck_estudiante_correo CHECK (correo_institucional LIKE
'%_@_%._%')
);

CREATE TABLE habilidad (
    id_habilidad INTEGER GENERATED ALWAYS AS IDENTITY,
    nombre       VARCHAR(80) NOT NULL,
    categoria    VARCHAR(40) NOT NULL,
    CONSTRAINT pk_habilidad PRIMARY KEY (id_habilidad),
    CONSTRAINT uq_habilidad_nombre UNIQUE (nombre)
);

CREATE TABLE interes (
    id_interes INTEGER GENERATED ALWAYS AS IDENTITY,
    nombre     VARCHAR(80) NOT NULL,
    categoria  VARCHAR(40) NOT NULL,
    CONSTRAINT pk_interes PRIMARY KEY (id_interes),
    CONSTRAINT uq_interes_nombre UNIQUE (nombre)
);
```

```sql
-- PUBLICA (1:N): el autor es una FK obligatoria (participación total, (1,1))
CREATE TABLE publicacion (
    id_publicacion INTEGER GENERATED ALWAYS AS IDENTITY,
    id_estudiante  INTEGER      NOT NULL,   -- autor
    contenido      TEXT         NOT NULL,
    fecha_hora     TIMESTAMPTZ  NOT NULL DEFAULT now(),
    tipo           VARCHAR(20)  NOT NULL,
    CONSTRAINT pk_publicacion PRIMARY KEY (id_publicacion),
    CONSTRAINT fk_publicacion_autor FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT ck_publicacion_tipo
        CHECK (tipo IN ('pregunta', 'recurso', 'noticia', 'meme')),
    CONSTRAINT ck_publicacion_contenido CHECK (length(btrim(contenido)) > 0)
);

-- CREA (1:N): el creador es una FK obligatoria en GRUPO
CREATE TABLE grupo (
    id_grupo       INTEGER GENERATED ALWAYS AS IDENTITY,
    id_creador     INTEGER      NOT NULL,
    nombre         VARCHAR(100) NOT NULL,
    descripcion    VARCHAR(500),
    categoria      VARCHAR(20)  NOT NULL,
    fecha_creacion DATE         NOT NULL DEFAULT CURRENT_DATE,
    CONSTRAINT pk_grupo PRIMARY KEY (id_grupo),
    CONSTRAINT fk_grupo_creador FOREIGN KEY (id_creador)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT ck_grupo_categoria
        CHECK (categoria IN ('estudio', 'hackathon', 'club'))
);

-- ORGANIZA (1:N): FK obligatoria al organizador
-- ASOCIADO_A (1:N, parcial): id_grupo es NULL cuando el evento es independiente
CREATE TABLE evento (
    id_evento       INTEGER GENERATED ALWAYS AS IDENTITY,
    id_organizador  INTEGER      NOT NULL,
    id_grupo        INTEGER,
    nombre          VARCHAR(120) NOT NULL,
    descripcion     VARCHAR(500),
    fecha_hora_inicio TIMESTAMPTZ NOT NULL,
    fecha_hora_fin    TIMESTAMPTZ NOT NULL,
```

```
    ubicacion       VARCHAR(150) NOT NULL,
    tipo            VARCHAR(20)  NOT NULL,
    CONSTRAINT pk_evento PRIMARY KEY (id_evento),
    CONSTRAINT fk_evento_organizador FOREIGN KEY (id_organizador)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_evento_grupo FOREIGN KEY (id_grupo)
        REFERENCES grupo (id_grupo)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT ck_evento_tipo CHECK (tipo IN ('taller', 'social',
'estudio')),
    CONSTRAINT ck_evento_fechas CHECK (fecha_hora_fin > fecha_hora_inicio)
);

-- ########################################################################
-- 2. ENTIDAD DÉBIL: COMENTARIO
--    PK compuesta (id_publicacion, id_comentario) -> relación
identificadora
--    PERTENECE_A. ESCRIBE (1:N) se implementa con id_estudiante (autor).
-- ########################################################################

CREATE TABLE comentario (
    id_publicacion INTEGER     NOT NULL,
    id_comentario  INTEGER     NOT NULL,   -- clave parcial (consecutivo por
publicación)
    id_estudiante  INTEGER     NOT NULL,   -- autor del comentario
    contenido      TEXT        NOT NULL,
    fecha_hora     TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT pk_comentario PRIMARY KEY (id_publicacion, id_comentario),
    CONSTRAINT fk_comentario_publicacion FOREIGN KEY (id_publicacion)
        REFERENCES publicacion (id_publicacion)
        ON UPDATE CASCADE ON DELETE CASCADE,       -- entidad débil: muere
con su propietaria
    CONSTRAINT fk_comentario_autor FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT ck_comentario_id CHECK (id_comentario > 0),
    CONSTRAINT ck_comentario_contenido CHECK (length(btrim(contenido)) > 0)
);

-- Asigna automáticamente el consecutivo de id_comentario por publicación
CREATE OR REPLACE FUNCTION fn_comentario_consecutivo()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.id_comentario IS NULL OR NEW.id_comentario = 0 THEN
```

```
-- bloqueo por publicación para evitar duplicados en concurrencia
        PERFORM 1 FROM publicacion WHERE id_publicacion = NEW.id_publicacion
FOR UPDATE;
        SELECT COALESCE(MAX(id_comentario), 0) + 1
          INTO NEW.id_comentario
          FROM comentario
         WHERE id_publicacion = NEW.id_publicacion;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Para insertar sin indicar id_comentario, se relaja temporalmente el NOT
NULL
-- mediante el trigger BEFORE (el valor se calcula antes de validar
restricciones).
CREATE TRIGGER trg_comentario_consecutivo
    BEFORE INSERT ON comentario
    FOR EACH ROW EXECUTE FUNCTION fn_comentario_consecutivo();
-- La PK sigue garantizando que id_comentario nunca quede nulo.

-- ############################################################################
-- 3. RELACIONES M:N (tablas asociativas)
-- ############################################################################

-- SIGUE: relación recursiva ESTUDIANTE (seguidor) - ESTUDIANTE (seguido)
CREATE TABLE sigue (
    id_seguidor  INTEGER NOT NULL,
    id_seguido   INTEGER NOT NULL,
    fecha_inicio DATE    NOT NULL DEFAULT CURRENT_DATE,
    es_mentoria  BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT pk_sigue PRIMARY KEY (id_seguidor, id_seguido),
    CONSTRAINT fk_sigue_seguidor FOREIGN KEY (id_seguidor)
        REFERENCES estudiante (id_estudiante) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT fk_sigue_seguido FOREIGN KEY (id_seguido)
        REFERENCES estudiante (id_estudiante) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT ck_sigue_no_autoseguimiento CHECK (id_seguidor <> id_seguido)
);

CREATE TABLE tiene_habilidad (
    id_estudiante INTEGER     NOT NULL,
    id_habilidad  INTEGER     NOT NULL,
    nivel         VARCHAR(15) NOT NULL DEFAULT 'basico',
```

```
    CONSTRAINT pk_tiene_habilidad PRIMARY KEY (id_estudiante, id_habilidad),
    CONSTRAINT fk_th_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT fk_th_habilidad FOREIGN KEY (id_habilidad)
        REFERENCES habilidad (id_habilidad) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT ck_th_nivel CHECK (nivel IN ('basico', 'intermedio',
'avanzado'))
);

CREATE TABLE tiene_interes (
    id_estudiante INTEGER NOT NULL,
    id_interes    INTEGER NOT NULL,
    CONSTRAINT pk_tiene_interes PRIMARY KEY (id_estudiante, id_interes),
    CONSTRAINT fk_ti_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT fk_ti_interes FOREIGN KEY (id_interes)
        REFERENCES interes (id_interes) ON UPDATE CASCADE ON DELETE CASCADE
);

CREATE TABLE reacciona (
    id_estudiante  INTEGER     NOT NULL,
    id_publicacion INTEGER     NOT NULL,
    tipo_reaccion  VARCHAR(15) NOT NULL,
    fecha_hora     TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT pk_reacciona PRIMARY KEY (id_estudiante, id_publicacion),
    CONSTRAINT fk_reac_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT fk_reac_publicacion FOREIGN KEY (id_publicacion)
        REFERENCES publicacion (id_publicacion) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT ck_reac_tipo
        CHECK (tipo_reaccion IN ('me_gusta', 'me_encanta', 'util',
'divertido', 'triste'))
);

CREATE TABLE participa_en (
    id_estudiante INTEGER     NOT NULL,
    id_grupo      INTEGER     NOT NULL,
    fecha_ingreso DATE        NOT NULL DEFAULT CURRENT_DATE,
    rol           VARCHAR(15) NOT NULL DEFAULT 'miembro',
    CONSTRAINT pk_participa_en PRIMARY KEY (id_estudiante, id_grupo),
```

```
    CONSTRAINT fk_pe_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT fk_pe_grupo FOREIGN KEY (id_grupo)
        REFERENCES grupo (id_grupo) ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT ck_pe_rol CHECK (rol IN ('miembro', 'moderador',
'administrador'))
);

CREATE TABLE asiste (
    id_estudiante     INTEGER     NOT NULL,
    id_evento         INTEGER     NOT NULL,
    estado_asistencia VARCHAR(15) NOT NULL DEFAULT 'inscrito',
    fecha_registro    TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT pk_asiste PRIMARY KEY (id_estudiante, id_evento),
    CONSTRAINT fk_as_estudiante FOREIGN KEY (id_estudiante)
        REFERENCES estudiante (id_estudiante) ON UPDATE CASCADE ON DELETE
CASCADE,
    CONSTRAINT fk_as_evento FOREIGN KEY (id_evento)
        REFERENCES evento (id_evento) ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT ck_as_estado
        CHECK (estado_asistencia IN ('inscrito', 'confirmado', 'asistio',
'cancelado'))
);

-- ########################################################################
-- 4. ÍNDICES sobre claves foráneas (PostgreSQL no los crea automáticamente)
--    Las PK compuestas ya indexan la primera columna; se indexa la segunda.
-- ########################################################################

CREATE INDEX ix_publicacion_autor      ON publicacion (id_estudiante,
fecha_hora DESC);
CREATE INDEX ix_comentario_autor       ON comentario (id_estudiante);
CREATE INDEX ix_grupo_creador          ON grupo (id_creador);
CREATE INDEX ix_evento_organizador     ON evento (id_organizador);
CREATE INDEX ix_evento_grupo           ON evento (id_grupo);
CREATE INDEX ix_evento_inicio          ON evento (fecha_hora_inicio);
CREATE INDEX ix_sigue_seguido          ON sigue (id_seguido);
CREATE INDEX ix_th_habilidad           ON tiene_habilidad (id_habilidad);
CREATE INDEX ix_ti_interes             ON tiene_interes (id_interes);
CREATE INDEX ix_reacciona_publicacion  ON reacciona (id_publicacion);
CREATE INDEX ix_participa_grupo        ON participa_en (id_grupo);
CREATE INDEX ix_asiste_evento          ON asiste (id_evento);
```

```sql
-- ########################################################################
-- 5. DATOS DE PRUEBA MÍNIMOS (para validar restricciones y relaciones)
-- ########################################################################

INSERT INTO estudiante (nombre, apellido, correo_institucional, carrera,
semestre, fecha_nacimiento)
VALUES ('Ana',  'Gómez',  'ana.gomez@pascual.edu.co',  'Ingeniería de
Sistemas', 5, '2004-03-12'),
       ('Luis', 'Pérez',  'luis.perez@pascual.edu.co', 'Diseño
Gráfico',         3, '2005-07-01');

INSERT INTO publicacion (id_estudiante, contenido, tipo)
VALUES (1, '¿Alguien tiene apuntes de Bases de Datos?', 'pregunta');

INSERT INTO comentario (id_publicacion, id_estudiante, contenido) VALUES (1,
2, 'Yo te los comparto.');
INSERT INTO comentario (id_publicacion, id_estudiante, contenido) VALUES (1,
1, '¡Gracias!');   -- id_comentario = 2

INSERT INTO sigue (id_seguidor, id_seguido, es_mentoria) VALUES (2, 1,
TRUE);
INSERT INTO grupo (id_creador, nombre, categoria) VALUES (1, 'Estudio BD',
'estudio');
INSERT INTO participa_en (id_estudiante, id_grupo, rol) VALUES (1, 1,
'administrador'), (2, 1, 'miembro');
INSERT INTO evento (id_organizador, id_grupo, nombre, fecha_hora_inicio,
fecha_hora_fin, ubicacion, tipo)
VALUES (1, 1, 'Taller de SQL', now() + interval '2 days', now() + interval
'2 days 2 hours', 'Aula 301', 'taller');
INSERT INTO asiste (id_estudiante, id_evento) VALUES (2, 1);
```